

Market Simulator Using Generative Adversarial Networks for the European Investment Bank

Tunahan Gumuskaya
Department of Mathematics
Columbia University
NY, USA
tg2885@columbia.edu

Wanjing Zhang
Department of Mathematics
Columbia University
NY, USA
wz2612@columbia.edu

Bo Yu
Department of Electrical
Engineering
Columbia University
NY, USA
by2345@columbia.edu

Abstract

Generative adversarial networks (GANs) have garnered significant success in generating samples from high-dimensional probability measures but grapple with temporal dependence seen in time-series data. This challenge is magnified by the increased dimension of the target space from prolonged time-series data streams, potentially rendering generative modeling impractical. Our study, building upon previous work, undertakes a comprehensive exploration of GANs in the context of yield curve data. Earlier research integrated GANs with a mathematically robust path feature extraction, the signature of a path, offering a universal description for data streams. This integration led to the development of a novel metric, (conditional) Sig-WI, and a generator, the conditional AR-FNN, optimized for capturing the temporal dependence of time series. Validations on synthetic and empirical datasets revealed its superiority over conventional benchmarks in similarity and predictive capability.

In our current endeavor, we identify and rectify oversights in code evaluation, and embark on an in-depth exploration of loss functions, pinpointing Mean Absolute Error as the paramount performer. Incorporating yield curve data from the European Investment Bank, we dissect yield rate data across periods, unearthing insights like mean-reversion in 30-year data. Through the use of the Autocorrelation Function, Partial Autocorrelation Function, and additive decomposition, we discerned underlying data characteristics. Further enhancing our approach, we systematized data representation and discovered the varying performance of GANs across all the combinations of yield curve maturities, with SigCWGAN standing out in consistency. Our methodologies expanded to time series classification techniques, with a focus on Dynamic Time Warping (DTW) and K-Means clustering for yield data, categorizing it into six unique clusters.

Key Links

- [Slides and link](#)
- [Report name and link](#)
- [Github link \(master branch\)](#)
- Data location and links

1. Introduction

1.1 Background

Established in 1958 and headquartered in Luxembourg, the European Investment Bank (EIB) stands as a nonprofit institution within the European Union (EU) that wields considerable influence in the financial sector, especially in the domain of bonds. Beyond its core function of extending financial support and loans to businesses and projects, the EIB actively engages with the bond market to secure the capital required for its operations and endeavors.

With a primary focus on advancing integration, fostering balanced development, and nurturing economic and social cohesion among EU member states, the EIB strategically employs the bond market to generate funds for its financial undertakings. This involves the issuance of bonds to a diverse array of investors, ranging from institutional entities and banks to individuals. These bonds function as debt securities, symbolizing loans from investors to the EIB, complete with fixed interest rates and predetermined maturity periods.

The EIB's foray into the bond market offers a multitude of advantages. It capitalizes on the market's liquidity and depth, effectively tapping into a global pool of investors. This diversification minimizes reliance on any one source of funding while yielding substantial capital to support overarching EU goals, including economic development, environmental sustainability, and social cohesion.

However, the EIB's role in the bond market transcends mere self-financing. The institution functions as a pivotal catalyst and issuer of bonds that further the cause of sustainable finance within the EU. By issuing green bonds, social bonds, and sustainability bonds, the EIB champions eco-friendly projects, social welfare initiatives, and investments aligned with the United Nations Sustainable Development Goals (SDGs). These bonds serve as beacons for investors committed to endorsing sustainable and socially responsible ventures while bolstering the EIB's financial resources.

Moreover, the EIB's influence extends beyond EU borders. Collaborative efforts with non-EU countries and financial institutions facilitate bond issuances that advance development and economic cooperation. Through cross-border bond transactions, the EIB fosters international collaboration and contributes to sustainable development initiatives in regions outside the EU.

In summary, the EIB's engagement with the bond market constitutes an integral facet of its financial operations. Through bond issuances, the institution raises the necessary funds for its lending endeavors and initiatives aimed at bolstering European economic development, environmental sustainability, and social cohesion. Additionally, the EIB's active role in the bond market drives the evolution of sustainable finance within the EU and promotes global cooperation through cross-border bond initiatives.

1.2 Motivation

Financial time series are characterized by non-stationarity and, due to tail risk events, it is often difficult for the same set of financial data to share the same joint distribution, making it difficult for ordinary robust training models to capture its complete characteristics. Because of this, financial researchers are often hampered in their prediction and judgment of risk events. Therefore, synthesizing feasible data with the same characteristics as real data is very urgent in the field of finance. In our research, we are committed to designing a complete algorithm for generating synthetic data by utilizing the outstanding ability of GAN in capturing data features and the contribution of rough path theory in constructing continuous data. Building on Lukasz Szpruch et al.'s work, we seek to further refine the algorithm, including tuning the parameters, understanding the data and nesting the features, and incorporating stage transitions into the metrics system for data construction. Fortunately, we have made good progress.

1.3 Previous Work and Our Contribution

A major portion of our efforts was dedicated to scrutinizing the research paper. We took a deep dive into the intricate details, methodologies, and the underlying principles presented. This was complemented by a meticulous examination of the code shared by the preceding groups.

Parallely, our team ventured into the fascinating domain of Generative Adversarial Networks (GANs). We embarked on a comprehensive exploration of how GANs can be adeptly applied to time series data. This exercise not only broadened our understanding of deep learning techniques but also helped us identify potential avenues and innovations we could introduce in our endeavor.

Recognizing the importance of a robust computational foundation, we prioritized the setting up of our infrastructure. Leveraging the versatile offerings of the Google Cloud Platform (GCP), we established a virtual machine instance tailored to our needs. This wasn't merely a technical setup; it was the creation of a conducive environment, equipped with deep learning capabilities, to facilitate smooth and efficient research in the coming weeks.

Lastly, we combined and compared data from different years, and analyzed the stationarity and other characteristics of the data one by one to gain a deeper understanding of the reasons behind the model results. We conducted in-depth mining of data features and utilized regime classification and clustering algorithms to identify

turning points in the data, and applied them to the generation of GAN model synthesized data, laying the groundwork for future research.

1.4 Datasets

In the early-stage experiments with multiple loss functions and other parameters, we worked on the dataset of the S&P 500 index (SPX) and Dow Jones index (DJI) and their realized volatility, which is retrieved from the Oxford-Man Institute's "realised library"[14]. Afterwards, we also utilized the dataset of the yield rate from the European Central Bank: <https://data.ecb.europa.eu/data/datasets/YC/data-information>. Detailed experiments and their source data will be illustrated in the following parts explicitly.

2. GAN Networks

2.1 Conditional Sig-Wasserstein GANs

Conditional Sig-Wasserstein Generative Adversarial Networks (Sig-Wasserstein GANs) have made significant advances in both the discriminator and generator components[2]. On the discriminator side, a novel (conditional) Sig-W1 metric has been developed that captures distributions over the path space well, and thus generalizes well to other features in time series analysis, such as Autocorrelation Functions (ACFs); moreover, the Sig-W1 metric possesses a unique signature feature space, thus eliminating the high computational cost associated with the classical Wasserstein metric. For the generator, an innovative approach involving the construction of an autoregressive feedforward neural network (AR-FNN) is used, which employs past paths and noise vectors to generate a random variable whose conditional distribution of the next prediction is as close as possible to the probability of the next observation given the previous data. With this structure, the generator effectively captures the autoregressive nature of the time series, allowing it to generate series that exhibit time-dependent and realistic trends.

The combined use of the discriminative power of the (conditional) Sig-W1 metric in the discriminator and the ability of the AR-FNN generator to capture temporal dynamics further enhances the effectiveness and accuracy of the conditional Sig-Wasserstein GAN.

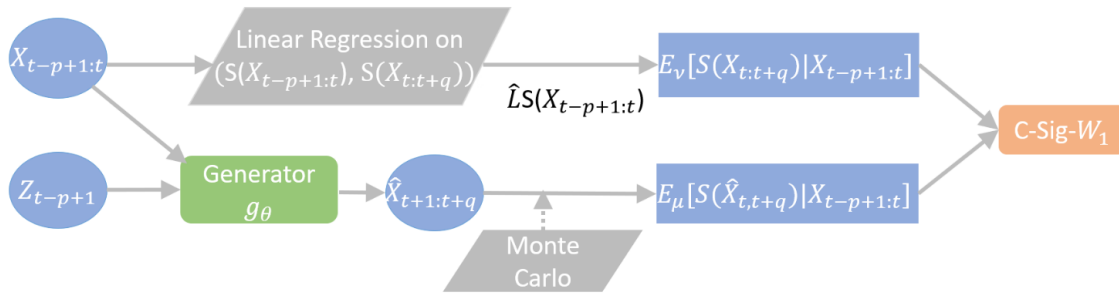


Figure 1. The illustration of the flowchart of SigCWGAN.

2.2 TimeGAN

TimeGAN, introduced by Yoon et al.[3] in 2019 from the University of California, Los Angeles, combines unsupervised GAN flexibility with controlled modeling of conditional temporal dynamics. Comprising four key components (embedding function, recovery function, sequence generator, and sequence discriminator), TimeGAN jointly trains auto-encoding and adversarial components to generate time series data that captures statistical fidelity and realistic trends. It enables the generation of synthetic time-series sequences mirroring real data, particularly valuable when genuine data is scarce. TimeGAN proves powerful for generating realistic time-series data, making it a valuable tool in domains reliant on time-series analysis, especially when real-world data is limited.

2.3 RCGAN

Esteban et al.[4] made a significant contribution to generating real-valued multi-dimensional time series, especially in the medical equipment context, with their Recurrent GAN (RGAN) and Recurrent Conditional GAN

(RCGAN) models. Both models employ recurrent neural networks within the traditional GAN architecture. RGAN generates realistic real-valued time series sequences, while RCGAN enhances it by incorporating conditional inputs. They introduced innovative evaluation methods, Maximum Mean Discrepancy (MMD) and "Train on Synthetic, Test on Real" (TSTR), which demonstrate the efficiency of using RCGAN-generated data for training supervised models with minimal performance reduction on real data tests. These evaluation methods underscore RCGAN's robustness and effectiveness, showing its potential for various practical applications, particularly in the medical field. Esteban et al.'s work represents a significant advancement in generating real-valued multi-dimensional time series and provides practical solutions for realistic data generation and GAN model evaluation.

2.4 GMMN

Hofer et al.[5] explore multivariate time-series modeling using generative neural networks to model joint innovation distribution. They present a framework that incorporates a GMMN-GARCH (Generalized Method of Moments Neural Network - Generalized Autoregressive Conditional Heteroskedasticity) approach, outlining key steps. They initially use ARMA-GARCH models for serial dependence in univariate marginal time series and employ PCA for dimension reduction in large sets of marginal time series. Cross-sectional dependence is effectively modeled using GMMN, a significant contribution. The paper's applications include yield curve modeling and foreign exchange-rate return analysis, demonstrating the practicality and effectiveness of the GMMN-GARCH approach. The strong performance of this model makes it a logical benchmark for comparison, signifying promise for its efficacy and reliability.

3. Implementation

This part of the documentation delineates our comprehensive journey of setting up and deploying the computational model. It initiates with the deployment of a Virtual Machine (VM) on the Google Cloud Platform (GCP), emphasizing specifics like machine types, GPU configurations, and disk image selections. An integral segment is the configuration of our environment, where the spotlight is on the usage of Conda – a robust package management system. The discussion later pivots to integrating Google Cloud Platform with Conda through a .yaml file and setting up a GitHub repository for efficient code management. A pivotal aspect of this process is data management, which involves structuring our dataset folder and understanding the content of the datasets utilized. In the subsequent sections, we delve into the nuances of training our GAN (Generative Adversarial Network) model. This encompasses experimenting with multiple loss functions to derive optimal results, making necessary adjustments to assimilate new yield data, and finally, the systematic tuning of hyperparameters using a tailored shell script to enhance model performance.

One of the most important parts to be acknowledged for implementation is the branches for the GitHub repository. The main branch is the working baseline for the original paper. New-data is the branch with the loss functions and added the new data for yield rates. EIB-TSC is the branch with the newly added time series classification label. EIB-TSC contains our latest updates while other branches also contain some notable changes. Please feel free to reach out to us for the details.

3.1 Deploying Virtual Machine and Environment

We embarked on deploying a virtual machine (VM) on the Google Cloud Platform (GCP), beginning with account setup and credit redemption. Post which, we created a VM instance in the "us-west3-b" zone, using specific machine types and GPU configurations. In our setup, we utilized the "ecbm4040-imageforstudents-tf24" disk image with firewall settings enabled for both HTTP and HTTPS. The next phase was dedicated to environment configuration, emphasizing the use of Conda for creating isolated project environments. We downloaded Miniconda tailored for Linux x86/64 architecture and installed it, post which we initialized the Conda environment. To integrate GCP with Conda, we made use of a .yaml file. During this configuration, we made necessary modifications to package versions to ensure system compatibility. This structured setup paves the way for optimized computational operations and tasks. Below is the detailed instructions.

3.1.1 Deploying Virtual Machine on GCP

Before deploying a virtual machine ("VM") on Google Cloud Platform, we first need to sign up for an account and enter our payment details. Then we redeemed our education credits using the link here:

<https://cloud.google.com/billing/docs/how-to/edu-grants#redeem>. Once that was done, we navigated to the "Compute Engine" tab on the left-hand menu and chose "Create Instance" under "VM instances". Before setting up the VM, we made sure to check GPU availability across different regions. We also noticed that on Google Cloud Platform, the GPU and machine types vary by region, with certain GPUs only being compatible with specific machine types.

Below is the information of our VM instance created:

- Machine Type: n1-standard-2
- Zone: us-west3-b
- Architecture: x86/64
- GPU: 1 x NVIDIA T4
- Disk Image: ecbm4040-imageforstudents-tf24
- Disk Size: 100GB
- Firewall Setting: On for HTTP and HTTPS

The methods for retrieving the disk image is inside the tutorial provided for setting up the VM. Alternatively, you can also choose to use the default deep learning image provided by Google Cloud Platform since this project uses PyTorch and the disk image provided integrates TensorFlow.

3.1.2 Configure Environment

Setting up a robust computational environment is pivotal to ensure streamlined execution of our tasks. An essential tool that assists in creating our environment is Conda. Conda is an open-source package management system that aids in setting up multiple environments for different projects, ensuring that software dependencies do not interfere with one another. In this section, we outline the steps to set up Conda and subsequently configure Google Cloud Platform (GCP) with a .yaml file.

1. Downloading Miniconda:

Begin by downloading the Miniconda script, which will provide us with the minimal installer for Conda. The following command is tailored for Linux x86_64 architecture:

```
wget
https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.
sh
```

2. Installation of Miniconda:

Once the download completes, execute the script to begin the installation process:

```
bash Miniconda3-latest-Linux-x86_64.sh
```

Follow the prompts as they appear. By the end of this step, Conda should be successfully installed on your system.

3. Activating Conda:

After installing Miniconda, it's essential to initialize the environment to activate Conda. This ensures that the Conda environment and the packages installed within it are available for use:

```
source ~/.bashrc
```

4. Verification of Installation:

To ensure that Conda has been installed correctly and is functioning as expected, check its version:

```
conda --version
```

If the installation was successful, this command will return the current version of Conda.

5. Setting up Google Cloud Platform (GCP) with .yaml File:

GCP configurations often use .yaml files to specify setups and dependencies. To set up GCP within the Conda environment:

```
conda env create -f eib_old_final.yaml
```

It is crucial to highlight that the 'signatory' package version was modified to be compatible with our local Linux machine. Therefore, before replicating this setup on different platforms, it is advisable to validate the versions of the respective packages to ensure compatibility.

In conclusion, setting up Conda and integrating it with GCP enhances the productivity and efficiency of computational tasks. Properly following the aforementioned steps ensures a seamless and error-free environment.

3.1.3 Setup GitHub repository and load data into VM

We initiated a connection to GitHub using an SSH key pair generated on our local machine. After

generating the key, we opened and copied the public key, subsequently adding it to our GitHub account's SSH settings. This was followed by cloning the specific repository onto our machine. Furthermore, we downloaded the necessary data and then uploaded the zipped file to our VM, either via a web-based SSH connection or GCP commands. Lastly, we relocated the .zip file to the designated code folder and unzipped it, ensuring the correct path adjustments where necessary.

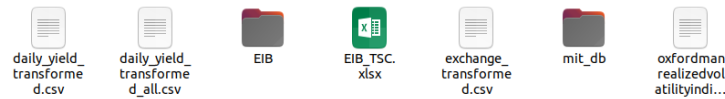
This enabled us to update our code on the VM and edit it on our own laptop simultaneously.

3.2 Data Loading

3.2.1 Folder Structure

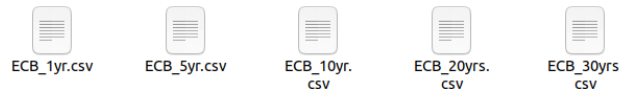
The dataset folder consists of two parts, one is the dataset from previous experiments, the other is datasets newly added for the experiments which includes the yield rates and the time series classification labels.

The structure is as follows:



3.2.2 EIB and EIBTSC Datasets

The structure within the EIB dataset is as follows.



3.3 Training GAN Network

3.3.1 Experiment with Multiple Loss Functions

In order to enhance the efficacy of hyperparameter tuning, a deliberate strategic decision was taken to curtail the number of training epochs from 1000 to 200. This decision was predicated upon empirical observations indicating that, beyond the threshold of 200 epochs, incremental alterations to the loss function exhibited negligible influence across the entirety of benchmark scenarios.

The principal objective encompassed an evaluative appraisal of diverse loss functions. To this end, a total of seven distinct loss functions were assimilated, subsequently furnishing the basis for a comprehensive comparative analysis. Notable among these were the Wasserstein Distance, the Adversarial Loss for Congan, Hinge_Gan, Mean Squared Error, Mean Absolute Error, Kullback-Leibler Divergence, and Binary Cross Entropy. The Wasserstein Distance measures the optimal "cost" of transforming one distribution into another. The Adversarial Loss for Congan represents the adversarial training process in a generative adversarial network (GAN), guiding the generator to produce data that is indistinguishable from real data to a discriminator. Hinge_Gan is a variant of GAN that employs a hinge loss to enhance training stability. Mean Squared Error quantifies the average squared differences between generated and actual data points. Mean Absolute Error similarly measures the average absolute differences between corresponding data points. Kullback-Leibler Divergence gauges the difference between two probability distributions. Binary Cross Entropy assesses the dissimilarity between predicted and actual binary outcomes, often used in classification tasks.

To facilitate the assimilation and synthesis of outcomes emerging from the seven disparate loss functions, a shell script was devised. This script meticulously executed a looping construct, sequentially traversing each loss function. The structure of the script was modified from the script that was introduced below at the 4.3.3 section. The resultant effect of this was the systematic accumulation of outcomes, thereby expediting the streamlined generation of exhaustive findings.

The assessment itself was confined exclusively to SPX data, encompassing the application of eight distinct loss functions. Within this ambit, outcomes were successfully derived for six of these loss functions. Regrettably,

endeavors to procure outcomes for the Kullback-Leibler Divergence and Binary Cross Entropy loss functions were beset by challenges.

Upon scrutinizing the derived results, it becomes discernible that the Mean Absolute Error (MAE) loss function emerged as the preeminent performer amidst the spectrum of evaluated alternatives. Closely trailing, the original researcher-developed loss function staked its claim as the second-most viable choice. Concomitantly, the Mean Squared Error (MSE) loss function garnered the third position in the hierarchy of performance. Regrettably, the residual loss functions yielded outcomes that were incongruent and non-comparable within the purview of our analysis.

3.3.2 Experiments on New Yield Data

Here is the link to the introduction of the dataset of the yield rate from the European Central Bank: <https://data.ecb.europa.eu/data/datasets/YC/data-information>. The primary content of the dataset includes daily prices and yields of government bonds, sourced from EuroMTS, along with ratings provided by Fitch Ratings. All data points are updated and published on a daily basis, with the base reference period being the previous day. In terms of methodology, the European Central Bank (ECB) ensures the data's validity through automated quality assessment checks. If one wishes to understand the dataset's naming conventions and other related details, they should refer to the YC underlying DSD (ECB_FMD2) that the ECB maintains. The overarching title for this dataset is "Financial markets data - Yield Curve (YC)", and its primary source is the European Central Bank (ECB).

In order to utilize the data, we made some necessary modifications to the GAN's training pipeline. The specific aspects can be found if you follow the training process. The key aspects in adding the feature is to remember to add the newly deployed dataset into hyperparameters.py file to fine tune your network for each dataset.

In order to test the hyperparameters, we also added an additional layer to the result folder that contains the hyperparameters used for the network, so that the result folder structure is different from the original paper's code structure.

3.3.3 Implementation of a Shell Script for Tuning Hyperparameters

In order to tune the hyper parameters with efficiency, we developed this script for running multiple sets of trainings automatically. Here is the script:

```
#!/bin/bash

# Define parameter ranges
p_range=(2 3 4)
q_range=(2 3 4)
dims_range=("3*(50,)" "4*(50,)" "4*(50,)"")

# Loop over parameter combinations
for p in "${p_range[@]"; do
    for q in "${q_range[@]"; do
        for dims in "${dims_range[@]"; do
            echo "Executing training command with p=$p, q=$q,
hidden_dims=$dims"
            python train.py -use_cuda -total_steps 1 -p $p -q $q -hidden_dims
$dims -datasets 'STOCKS' -algo 'SigCWGAN' 'GMMN'
        done
    done
done
Done
```

This shell script is designed for tuning hyperparameters in a machine learning training scenario. It employs a nested loop structure to iterate through different combinations of hyperparameters for a training command. The parameters being tuned are 'p', 'q', and 'hidden_dims'. The script initializes arrays 'p_range', 'q_range', and

'dims_range' that hold possible values for each parameter. The nested loops iterate through all combinations of 'p', 'q', and 'hidden_dims' values, printing a message indicating the current parameter values being evaluated. Inside the loop, training script ('train.py') is invoked with the specified hyperparameters, including the options to use CUDA for acceleration, perform only one training step ('total_steps 1'), and set the 'p', 'q', 'hidden_dims', 'datasets', and 'algo' parameters accordingly. The latter parameters represent dataset names ('STOCKS') and algorithm choices ('SigCWGAN' and 'GMMN'). This script facilitates an automated search over hyperparameter combinations for training our GAN.

4. Results

4.1 Evaluating SPX Data: Discoveries, Challenges, and the Superiority of MAE Loss Function

Our focused testing on SPX data revealed intriguing insights. While we succeeded in obtaining results for six of the eight loss functions, challenges arose with the Kullback-Leibler Divergence and Binary Cross Entropy functions. Nevertheless, our analysis bore fruit: the Mean Absolute Error (MAE) loss function emerged as the top performer, closely followed by the original researcher-developed function. Rounding off our top three was the Mean Squared Error (MSE). However, it became apparent that the other functions couldn't match the performance benchmarks set by our top contenders.

Loss Function for SigCWGAN	seed	r2_tstr	r2_trtr	predictive_score	abs_metric	acf_id_lag=1	kurtosis	skew	cross_correl	sig_w1_metri	assets
Original	seed=0	0.35783	0.36782	0.00998	0.01043	0.03379	1.87859	0.15575	0.0197	0.10038	SPX
Wasserstein distance	seed=0	-4857917.23	0.36782	4857917.597	0.08672	0.46734	inf	4.0987	1.17592	inf	SPX
Adversarial Gogan	seed=0	-60629.0292	0.36782	60629.39704	0.08675	0.52903	5.21142	1.81808	1.17592	inf	SPX
Hinge Gan	seed=0	-25.96418	0.36782	26.332	0.09015	0.36994	22.67845	4.04326	0.85086	66.06558	SPX
Mean Squared Error	seed=0	0.23366	0.36782	0.13416	0.01037	0.17122	2.64	0.03534	0.0191	0.14066	SPX
Mean Absolute Error	seed=0	0.36166	0.36782	0.00616	0.01419	0.03172	0.86224	0.05974	0.01875	0.10346	SPX

Figure 2. Comparisons of different loss functions. Best performers for each of the metrics are highlighted in yellow.

4.2 Comprehensive Analysis of Yield Rate Data: Comparison Between “return” and “change”

Our approach to the yield rate data was systematic and comprehensive. Spanning periods of 1, 5, 10, 20, and 30 years, we dissected each dataset meticulously. To add depth to our data, we engineered the "return" and "change" columns from the yield rates. While the "return" column showed inconsistencies, especially near zero rates, the "change" column provided more stable insights. The augmented Dickey-Fuller test was instrumental in assessing stationarity. It confirmed our suspicions regarding mean reversion in both the yield rate return and change columns. The Hurst exponent was another invaluable tool, revealing unique trends across the various time frames, with the 30-year data exhibiting mean-reverting properties – a key insight.

Our decision to employ both ACF and PACF to uncover significant lags proved beneficial. These statistical tools unveiled lags that bore strong correlations, providing us a pathway to improve our model's efficiency. Lastly, through additive decomposition, we successfully distinguished between the seasonality, trend, and residuals inherent in our data, granting us a clearer perspective.

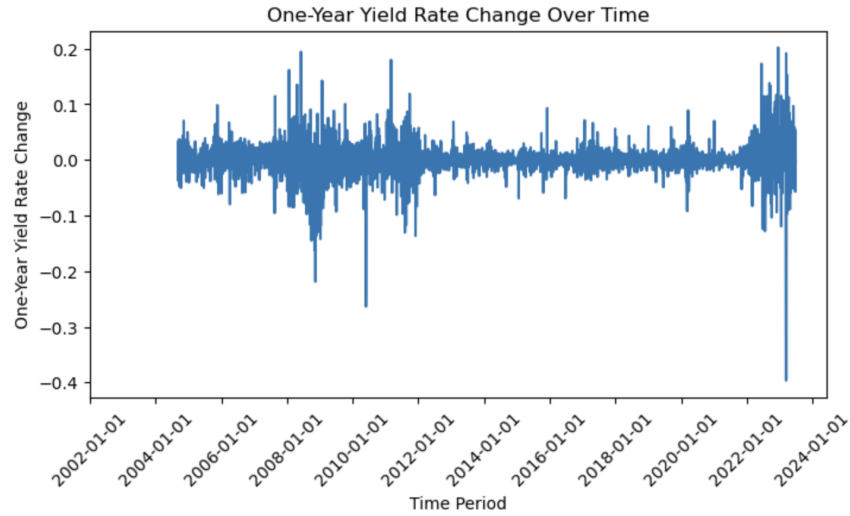


Figure 3. Change of one-year yield rate over time.

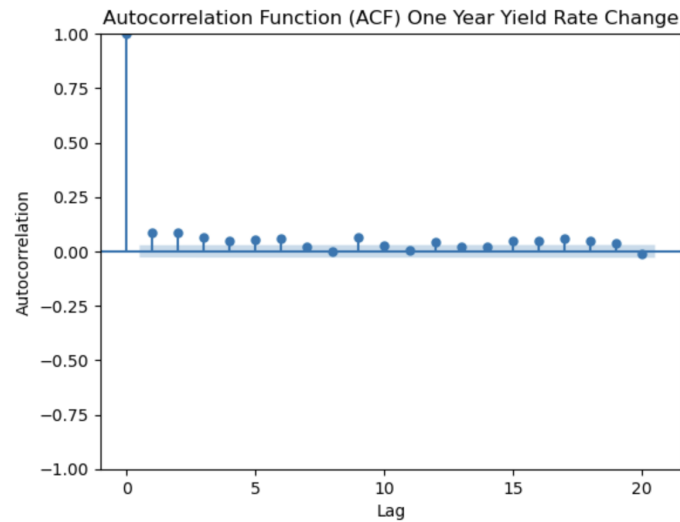


Figure 4. Result of the ACF test.

To assess stationarity using the yield rate column, we utilized the Hurst exponent. The results yielded intriguing findings:

- The 1-year data exhibited a trending behavior.
- The 5-year data also displayed a trending pattern.
- The 10-year data followed a Geometric Brownian Motion.
- Similarly, the 20-year data demonstrated characteristics of Geometric Brownian Motion.
- Notably, the 30-year data exhibited mean-reverting properties.

To gain a deeper understanding of the data, we employed additive decomposition to examine the presence of seasonality, trend, and residuals. This decomposition technique allowed us to separate the data into its constituent components.

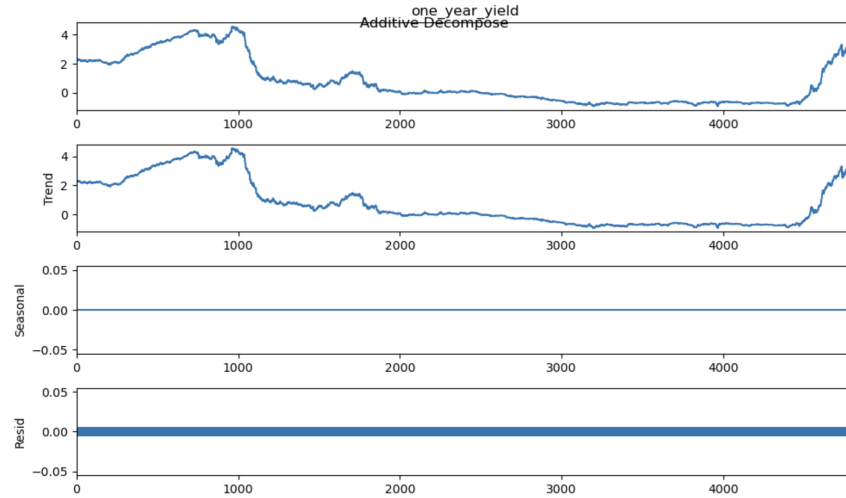


Figure 5. Examination of the presence of seasonality, trend and residuals.

4.3 Understanding the Input and Output Data

As we journeyed deeper into our investigation, there emerged a clear need for a more organized representation and exportation of our data. Responding to this, our team worked diligently to improve the code, ensuring that both input and output data could be efficiently exported to distinct Excel files. This not only facilitated better data management but also simplified the analysis and interpretation of the data.

a) Input Data:

Our input data's complexity is underscored by its two-dimensional nature for each yield curve maturity. The first dimension, a normalized measure of the 'YieldRateChange', offers insights into the fluctuation from the mean. The second dimension, obtained through the Exponentially Weighted Moving Average with a lambda of 0.94, provides a smoothed representation of the yield rate changes over time.

First Dimension: $(\text{YieldRateChange}(t) - \text{YieldRateChange_Mean}) / \text{YieldRateChange_std}$

Second Dimension: Exponentially Weighted Moving Average using the $\lambda = 0.94$.

Adding another layer of granularity, we've incorporated five lag versions for each variable. Considering we're operating with two dimensions, this equates to ten lag variables for each maturity.

	A	B	C	D	E	F	G	H	I	J	K	L
1	0	1	2	3	4	5	6	7	8	9	10	11
2	0.989102	-0.59946	1.008116	-0.53452	1.019362	-0.47249	0.995524	-0.41628	0.971848	-0.36549	0.994384	-0.3158
3	1.008116	-0.53452	1.019362	-0.47249	0.995524	-0.41628	0.971848	-0.36549	0.994384	-0.3158	0.987218	-0.26972
4	1.019362	-0.47249	0.995524	-0.41628	0.971848	-0.36549	0.994384	-0.3158	0.987218	-0.26972	0.988374	-0.22629
5	0.995524	-0.41628	0.971848	-0.36549	0.994384	-0.3158	0.987218	-0.26972	0.988374	-0.22629	0.995123	-0.18489
6	0.971848	-0.36549	0.994384	-0.3158	0.987218	-0.26972	0.988374	-0.22629	0.995123	-0.18489	0.978294	-0.14743
7	0.994384	-0.3158	0.987218	-0.26972	0.988374	-0.22629	0.995123	-0.18489	0.978294	-0.14743	0.977938	-0.11225
8	0.987218	-0.26972	0.988374	-0.22629	0.995123	-0.18489	0.978294	-0.14743	0.977938	-0.11225	0.984645	-0.0786
9	0.988374	-0.22629	0.995123	-0.18489	0.978294	-0.14743	0.977938	-0.11225	0.984645	-0.0786	0.984282	-0.047

Figure 6. Input data with five lags.

b) Output Data:

Emerging from the depths of our refined GAN model are synthetic outputs for both the first and second dimensions. But there's an additional layer: due to the hyperparameter p being set at 3, the model also synthesizes two extra lag variables. This enables our model to not only reproduce synthetic data akin to the real-world yield curve but also to understand and mimic its temporal patterns.

	A	B	C	D	E	F
1	0	0	0	0	0	0
2	1.025445	-0.42352	1.04296	-0.57479	0.965177	-0.24948
3	0.982699	-0.41712	1.028476	-0.34425	1.072527	-0.17914
4	1.017914	-0.2981	1.006201	-0.4307	0.961023	-0.10021
5	0.986483	-0.33529	1.021227	-0.25951	1.036127	-0.25379
6	0.95037	-0.28875	0.965749	-0.24987	1.019569	-0.03244
7	1.013034	-0.11397	0.957888	-0.16441	0.986067	-0.08578
8	0.960866	-0.21858	0.960151	-0.09452	0.992985	-0.01167
9	1.053741	-0.19342	0.968644	0.028423	1.040388	-0.09621

Figure 7. Output data under $p=3$

4.4 Performance Evaluation Metrics for Time Series Analysis on Yield Curve Maturities

In our quest to understand and evaluate the intricacies of time series data, especially in the context of yield curves, it becomes imperative to deploy a variety of metrics. These metrics not only quantify various characteristics of the data but also serve as touchpoints to compare synthetic outputs with real data.

1. **Temporal Correlations:** A pivotal metric, it gauges the interrelationships among successive data points within a sequence. In essence, it offers insights into how values evolve over time.
2. **Feature Correlations:** Offering a broader perspective, this metric sheds light on inter-relationships between different variables within the dataset, emphasizing concurrent behaviors and dependencies.
3. **r2_tstr and r2_trtr:** The 'r2_tstr' is indicative of how well a model trained on synthetic data performs on real data. Conversely, 'r2_trtr' evaluates the performance of a model trained and tested on real data. A derivative metric, the predictive_score, is simply the difference between the two, showcasing the divergence in performance between real and synthetic models.
4. **abs_metric:** Delving deeper into the distribution of the time series, this metric focuses on the empirical probability density function (epdf) of both real and synthetic datasets. The absolute disparity between the two epdfs provides insights into the marginal distribution variations.
5. **acf_id_lag=1:** Serving as a mirror to the linear interdependence of a data point with its predecessor, the autocorrelation at Lag 1 measures the consistency and pattern between successive data points.
6. **kurtosis & skew:** Both of these metrics shed light on the nature of the distribution of the dataset. While kurtosis pinpoints outliers and extreme values, skewness quantifies the asymmetry of the distribution, giving insights into potential left or right biases.
7. **cross_correl:** Acting as a temporal magnifying glass, cross-correlation quantifies the similarity between two time series over varied time lags. This metric is invaluable when trying to discern time delays between series or spotting patterns within longer time series sequences.
8. **sig_w1_metric:** A mathematical object that captures the ordered iterated integrals of a path. It provides a multi-level summary of the path's behavior, encapsulating not just where the path goes, but also how it gets there.

Performance Metrics for Yield Curve Maturities: In our current project phase, we have applied these metrics to all of the yield curves with different maturities. The figure 8 illustrates only one-year and five-year maturity yield curves. The ensuing results provide a comprehensive view of the model's performance, with each metric offering distinct insights.

summary											
model_id	seed	r2_tstr	r2_trtr	predictive_score	abs_metric	acf_id_lag=1	kurtosis	skew	cross_correl	sig_w1_metric	durations
TimeGAN	1yr	0,99960	0,99985	0,00025	0,03229	0,00016	0,02314	0,00693	0,00106	0,00833	p=3_q=3_hidden_dims=(50, 50, 50)
RCGAN	1yr	0,99959	0,99985	0,00027	0,02950	0,00027	0,01176	0,00407	0,00089	0,00771	p=3_q=3_hidden_dims=(50, 50, 50)
SigCWGAN	1yr	0,99928	0,99985	0,00057	0,02971	0,00097	0,01661	0,00544	0,00022	0,00425	p=3_q=3_hidden_dims=(50, 50, 50)
GMMN	1yr	0,99955	0,99985	0,00030	0,03289	0,00049	0,08495	0,01688	0,00416	0,00944	p=3_q=3_hidden_dims=(50, 50, 50)
RCWGAN	1yr	0,99956	0,99985	0,00029	0,02897	0,00010	0,01790	0,00541	0,00099	0,00861	p=3_q=3_hidden_dims=(50, 50, 50)
TimeGAN	5yr	0,99817	0,99977	0,00160	0,05701	0,00131	0,09860	0,02973	0,02497	0,03792	p=3_q=3_hidden_dims=(50, 50, 50)
RCGAN	5yr	0,99806	0,99977	0,00171	0,06660	0,00124	0,08732	0,02765	0,03328	0,04389	p=3_q=3_hidden_dims=(50, 50, 50)
SigCWGAN	5yr	0,91057	0,99977	0,08920	0,09096	0,01892	0,16755	0,09553	0,08134	0,01009	p=3_q=3_hidden_dims=(50, 50, 50)
GMMN	5yr	0,99664	0,99977	0,00313	0,06692	0,00287	0,13717	0,04214	0,05999	0,04989	p=3_q=3_hidden_dims=(50, 50, 50)
RCWGAN	5yr	0,99922	0,99977	0,00055	0,04641	0,00068	0,03453	0,00671	0,01000	0,02704	p=3_q=3_hidden_dims=(50, 50, 50)

Figure 8. Numerical results of 1 year and 5 year maturities of yield curve datasets.

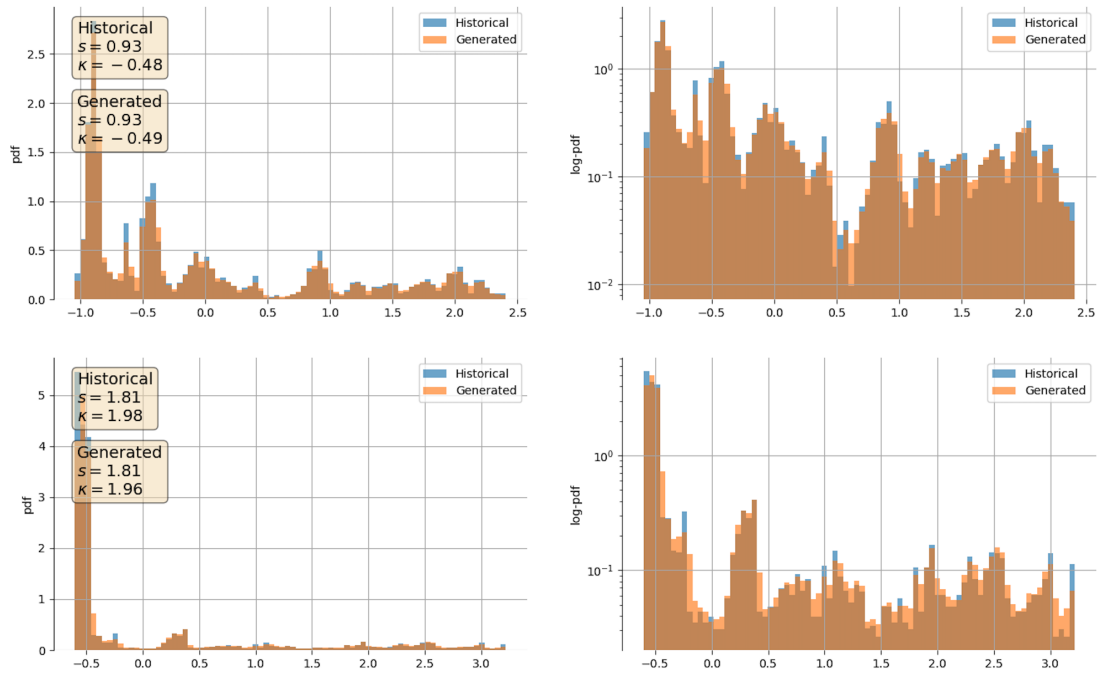


Figure 9. Comparison of the marginal distributions of the generated and real data for 1 and 5 year yield curve maturities.

We ran each of the yield curves separately (1 year only, 5 year only, etc.) and gathered the results. Once each maturity runs separately, results can show patterns, outliers, or specific behaviors for each tenor. This can provide insights into how different parts of the yield curve react to various factors.

model_id	seed	r2_tstr	r2_trtr	predictive_score	abs_metric	acf_id_lag=1	kurtosis	skew	cross_correl	sig_w1_metric	d
TimeGAN	1yr	0.99960	0.99985	0.00025	0.03229	0.00016	0.02314	0.00693	0.00106	0.00833	F
RCGAN	1yr	0.99959	0.99985	0.00027	0.02950	0.00027	0.01176	0.00407	0.00089	0.00771	F
SigCWGAN	1yr	0.99928	0.99985	0.00057	0.02971	0.00097	0.01661	0.00544	0.00022	0.00425	F
GMMN	1yr	0.99955	0.99985	0.00030	0.03289	0.00049	0.08495	0.01688	0.00416	0.00944	F
RCWGAN	1yr	0.99956	0.99985	0.00029	0.02897	0.00010	0.01790	0.00541	0.00099	0.00861	F
TimeGAN	5yr	0.99937	0.99968	0.00031	0.02328	0.00003	0.02197	0.00562	0.00215	0.00733	F
RCGAN	5yr	0.99931	0.99968	0.00038	0.02242	0.00002	0.00434	0.00198	0.00167	0.00877	F
SigCWGAN	5yr	0.99889	0.99968	0.00079	0.02455	0.00069	0.00270	0.00060	0.00015	0.00542	F
GMMN	5yr	0.99914	0.99968	0.00054	0.02689	0.00035	0.04532	0.01438	0.00556	0.01024	F
RCWGAN	5yr	0.99923	0.99968	0.00046	0.02326	0.00004	0.00742	0.00212	0.00231	0.00954	F
TimeGAN	10yr	0.99928	0.99969	0.00041	0.02465	0.00015	0.00265	0.00157	0.00254	0.00929	F
RCGAN	10yr	0.99930	0.99969	0.00039	0.02512	0.00011	0.00429	0.00048	0.00219	0.00864	F
SigCWGAN	10yr	0.99887	0.99969	0.00082	0.03100	0.00093	0.00347	0.00223	0.00122	0.00548	F
GMMN	10yr	0.99902	0.99969	0.00067	0.03477	0.00035	0.01121	0.00633	0.00446	0.01174	F
RCWGAN	10yr	0.99908	0.99969	0.00061	0.03229	0.00004	0.00594	0.00347	0.00224	0.01008	F
TimeGAN	20yr	0.99916	0.99964	0.00048	0.03174	0.00006	0.00054	0.00433	0.00232	0.00953	F
RCGAN	20yr	0.99915	0.99964	0.00049	0.02945	0.00009	0.00408	0.00620	0.00202	0.00963	F
SigCWGAN	20yr	0.99863	0.99964	0.00101	0.03833	0.00102	0.00341	0.00387	0.00177	0.00635	F
GMMN	20yr	0.99871	0.99964	0.00094	0.03472	0.00026	0.00732	0.00268	0.00471	0.01220	F
RCWGAN	20yr	0.99868	0.99964	0.00096	0.04151	0.00019	0.01101	0.00085	0.00450	0.01155	F
TimeGAN	30yr	0.99901	0.99954	0.00053	0.03374	0.00015	0.00107	0.00307	0.00188	0.00936	F
RCGAN	30yr	0.99895	0.99954	0.00059	0.03905	0.00015	0.00683	0.00228	0.00315	0.00961	F
SigCWGAN	30yr	0.99823	0.99954	0.00131	0.03331	0.00105	0.00455	0.00514	0.00242	0.00678	F
GMMN	30yr	0.99837	0.99954	0.00117	0.03929	0.00020	0.00877	0.00093	0.00523	0.01319	F
RCWGAN	30yr	0.99833	0.99954	0.00120	0.03868	0.00051	0.01443	0.00829	0.00354	0.01272	F

Figure 10. Numerical results of various years' yield curve datasets.

Figure 11 depicts how many times each model is the best performer for each of the evaluation metrics out of 5 Yield Curves (Maturities 1yr, 5yr, 10yr, 20yr, 30yr). While TimeGan outperforms for the predictive score metric, SigCWGAN outperforms for both cross_correlation and sig_w1 metric.

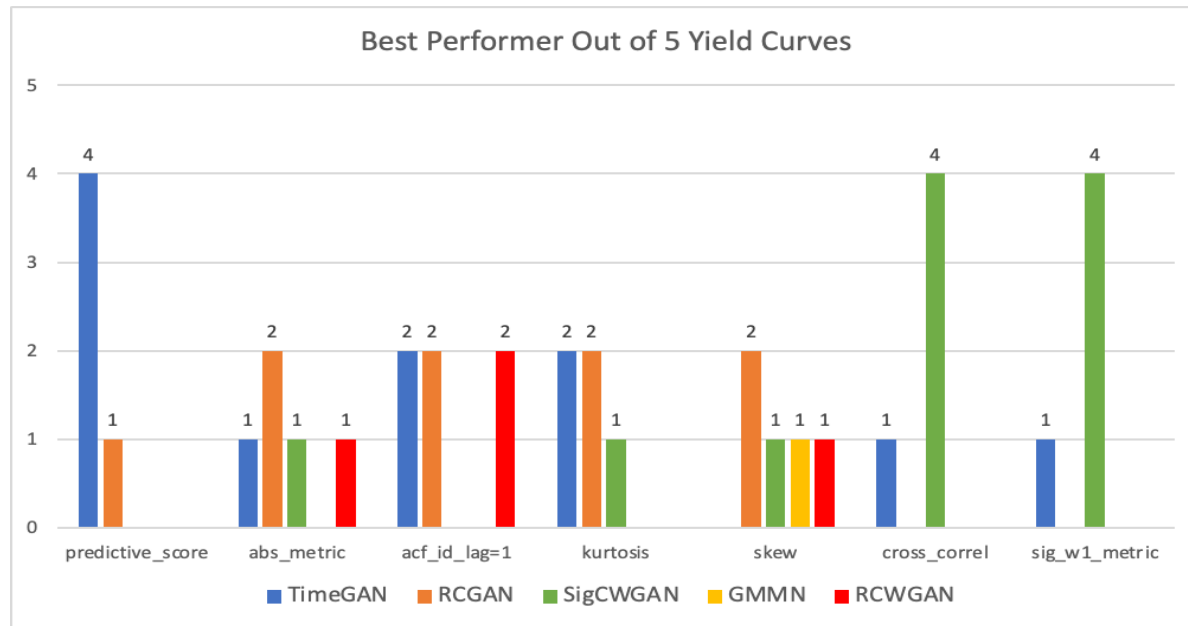


Figure 11. Comparisons of various years' yield curve datasets' performance under different metrics.

An In-Depth Examination of Generative Adversarial Networks Performance Across Multiple Yield Curve Maturities (1yr, 5yr, 10yr, 20yr, 30yr)

We had to adjust the code to be able to combine different yield curves. We did experiments using all permutations of data channels ranging from 1-year to 30-year intervals, and for all the metrics. Combining different yield curve maturities to run a GAN model offers several potential benefits and implications:

- **Richer Feature Set:** By integrating multiple yield curve maturities, we are essentially pooling together more features that can capture interdependencies between these maturities. This might provide a more comprehensive picture of the financial landscape.
- **Correlation Insights:** Yield curves often move in ways that reflect changing market expectations about future interest rates. By modeling different maturities together, the GAN can potentially learn and reproduce these subtle inter-maturity correlations, leading to more realistic synthetic data.
- **Higher Dimensionality:** By adding different yield curve maturities as new variables, we have increased the dimensionality of our input data. This expanded dimensionality can enable the model to capture a broader range of behaviors and relationships, but also may introduce challenges. Higher-dimensional models can sometimes be more prone to overfitting and may require more data to train effectively.
- **Potential for Generalization:** If the combined model is trained effectively, it may generalize better to unseen scenarios or subtle shifts in the financial landscape, as it has a more holistic view of the yield curve dynamics.

Figure 12 depicts the results for some of the combinations. The best performer of each yield curve combination for each evaluation metric is highlighted in yellow:

model_id	Data Channel	r2_tstr	r2_trr	predictive_score	abs_metric	acf_id_lag=1	kurtosis	skew	cross_correl	sig_w1_metric
TimeGAN	1yr	0.99960	0.99985	0.00025	0.03229	0.00016	0.02314	0.00693	0.00106	0.00833
RCGAN	1yr	0.99959	0.99985	0.00027	0.02950	0.00027	0.01176	0.00407	0.00089	0.00771
SigCWGAN	1yr	0.99928	0.99985	0.00057	0.02971	0.00097	0.01661	0.00544	0.00022	0.00425
GMMN	1yr	0.99955	0.99985	0.00030	0.03289	0.00049	0.08495	0.01688	0.00416	0.00944
RCWGAN	1yr	0.99956	0.99985	0.00029	0.02897	0.00010	0.01790	0.00541	0.00099	0.00861
TimeGAN	1yr_10yr	0.99862	0.99977	0.00115	0.06656	0.00109	0.04648	0.02049	0.02916	0.03993
RCGAN	1yr_10yr	0.99735	0.99977	0.00242	0.07670	0.00159	0.13131	0.04898	0.06557	0.04945
SigCWGAN	1yr_10yr	0.99871	0.99977	0.00106	0.04585	0.00137	0.01107	0.00352	0.00981	0.01267
GMMN	1yr_10yr	0.99788	0.99977	0.00189	0.06654	0.00207	0.06679	0.02485	0.06269	0.04384
RCWGAN	1yr_10yr	0.99913	0.99977	0.00064	0.05153	0.00089	0.02958	0.00669	0.02554	0.02971
TimeGAN	1yr_10yr_20yr	0.99208	0.99973	0.00765	0.09938	0.00180	0.19261	0.09203	0.18059	0.10222
RCGAN	1yr_10yr_20yr	0.99131	0.99973	0.00842	0.10674	0.00256	0.21686	0.09382	0.20145	0.11048
SigCWGAN	1yr_10yr_20yr	0.98360	0.99973	0.01613	0.08798	0.00409	0.02707	0.01383	0.18763	0.01607
GMMN	1yr_10yr_20yr	0.98356	0.99973	0.00617	0.07474	0.00501	0.09774	0.04965	0.19030	0.08151
RCWGAN	1yr_10yr_20yr	0.99814	0.99973	0.00159	0.06889	0.00206	0.06517	0.02145	0.06602	0.05626
TimeGAN	1yr_10yr_20yr_30yr	0.98579	0.99969	0.01390	0.12663	0.00328	0.32983	0.10733	0.70631	0.18850
RCGAN	1yr_10yr_20yr_30yr	0.98799	0.99969	0.01169	0.12007	0.00279	0.25435	0.09816	0.38583	0.15290
SigCWGAN	1yr_10yr_20yr_30yr	0.99878	0.99969	0.00091	0.05930	0.00287	0.00794	0.00300	0.03205	0.02317
GMMN	1yr_10yr_20yr_30yr	0.98965	0.99969	0.01003	0.08254	0.00341	0.17213	0.07550	0.41570	0.11553
RCWGAN	1yr_10yr_20yr_30yr	0.99779	0.99969	0.00190	0.07481	0.00210	0.02710	0.01399	0.08801	0.07300
TimeGAN	1yr_10yr_30yr	0.99140	0.99970	0.00830	0.09475	0.00224	0.17992	0.08230	0.15794	0.10243
RCGAN	1yr_10yr_30yr	0.99111	0.99970	0.00858	0.10606	0.00225	0.20169	0.09345	0.18940	0.10878
SigCWGAN	1yr_10yr_30yr	0.98890	0.99970	0.01080	0.08621	0.00319	0.02324	0.01367	0.13176	0.01645
GMMN	1yr_10yr_30yr	0.99126	0.99970	0.00844	0.07610	0.00471	0.09185	0.05438	0.22439	0.08839
RCWGAN	1yr_10yr_30yr	0.99794	0.99970	0.00175	0.06733	0.00239	0.05150	0.01261	0.05949	0.05925
TimeGAN	1yr_20yr	0.99876	0.99975	0.00099	0.06037	0.00089	0.03749	0.00822	0.03056	0.03888
RCGAN	1yr_20yr	0.99756	0.99975	0.00219	0.07164	0.00139	0.10373	0.04080	0.06476	0.04958
SigCWGAN	1yr_20yr	0.99877	0.99975	0.00098	0.05098	0.00241	0.00984	0.00565	0.02137	0.01318
GMMN	1yr_20yr	0.99758	0.99975	0.00217	0.05824	0.00169	0.05091	0.02236	0.07638	0.04203
RCWGAN	1yr_20yr	0.99896	0.99975	0.00078	0.04713	0.00086	0.01459	0.00663	0.00683	0.02691

Figure 12. Numerical results of various combinations of years' yield curve datasets.

Predictive Score: The predictive scores vary across different models and data channels. SigCWGAN generally exhibits higher predictive scores, especially with higher dimensional datasets, while RCWGAN tends to show better performance in many lower dimensional channels.

Absolute Metric: Absolute metrics are relatively consistent across various models. TimeGAN, RCGAN, and RCWGAN have more similar values, while SigCWGAN shows more fluctuations.

ACF (Autocorrelation Function) Id with Lag=1: This metric has lower values for most models, with RCWGAN frequently having the lowest values.

Cross-Correlation: SigCWGAN often demonstrates lower cross-correlation, especially in the channels involving 20-year and 30-year intervals. This indicates less similarity between the different time series.

SigCWGAN: There is a clear variance in this metric across different models and data channels. SigCWGAN has lower scores and outperforms every time.

General Trends across Time Intervals:

1-Year Intervals: Generally, metrics are lower and show less variation. **Extended Intervals (10yr, 20yr, 30yr):** There is a clear increase in most metrics, reflecting more complexity or possibly less accurate modeling. In conclusion, the various GAN models show differing performance across the different time intervals and metrics. Detailed analysis could further explore the relationships between these metrics and the underlying data insights to identify the best model for specific applications. Also, from the provided data, we can determine how many times each GAN model performed the best by looking at the lowest scores for each metric, as the problem statement indicates that lower scores are better.

A list the number of times each GAN performed the best across all the metrics (predictive_score, abs_metric, acf_id_lag=1, kurtosis, skew, cross_correl, and sig_w1_metric):

Model Name	Predictive Score	Abs Metric	ACF ID Lag-1	Kurtosis	Skew	Cross Correlation	Sig W1 Metric	Total
TimeGAN	5	1	12	3	1	1	0	23
RCWGAN	17	12	16	6	7	11	0	59
SigCWGAN	7	12	1	19	18	18	30	115
GMMN	0	0	0	1	2	0	0	3
RCGAN	1	3	4	1	2	0	0	11

Table 1. Comparison of different GAN models' performance for all the combinations of yield curves.

From this analysis, we can see that SigCWGAN performed the best the most times across the different metrics, followed by RCWGAN, TimeGAN, RCGAN, and GMMN.

4.5 Classifying Time Series Data

In the dynamic realm of financial analysis, understanding and categorizing distinct periods or regimes of time series data can be vital. These regimes, identified by specific patterns or statistical deviations, facilitate enhanced modeling and forecasting of future values.

Methodologies for Time Series Classification:

1. Change Point Detection:
 - Statistical methods including the Cumulative Sum (CUSUM) and likelihood ratio tests.
 - Bayesian Change Point Analysis.
 - Binary Segmentation.
2. Statistical Measures:
 - Volatility clustering employing techniques like GARCH.
 - Monitoring drastic shifts in skewness and kurtosis.
3. Hidden Markov Models (HMM):
 - Estimating hidden states or regimes that influence observed data sequences.
4. Clustering Algorithms:
 - Employing K-means clustering on rolling windows.
 - Utilizing hierarchical clustering for nested regime structures.
5. Wavelet Decomposition:
 - Segmenting the time series into varying frequency components.
6. Machine Learning Paradigms:
 - Incorporating decision trees or Random Forests with rolling window features.
 - Utilizing recurrent neural structures like LSTM or GRU.
7. Kalman Filters & Variants:
 - Implementing advanced filters like Extended Kalman and Particle Filters.
8. Structural Break Tests:
 - Applying the Bai-Perron breakpoint test.
 - The Chow test for recognizing structural deviations.
9. Trend & Seasonality Decomposition:
 - Deploying STL (Seasonal-Trend decomposition using LOESS).
10. Rolling Window Analysis:
 - Computing and visualizing statistics over rolling windows.
11. Fourier Analysis:
 - Decomposing time series into frequency constituents.

For our yield data classification, the intrinsic nature of time series necessitated a clustering mechanism that accounts for temporal alignments. To this end, Dynamic Time Warping (DTW) emerged as an ideal choice. Paired with the traditional K-Means clustering algorithm, DTW[7] accentuates the temporal features of our data.

We initiated our approach by applying an Exponentially Weighted Moving Average (EWM) to the original features. The EWM function smoothens our dataset, making it more amenable to clustering. The smoothing factor is encapsulated by the 'span' parameter, deduced from the equation:

	1Year	5Year	10Year	20Year	30Year
0	0.989117	1.340518	1.402158	1.474667	1.610982
1	1.007561	1.353877	1.402398	1.468492	1.602874
2	1.019012	1.371263	1.413516	1.473038	1.604719
3	0.996265	1.325551	1.374516	1.447163	1.584901
4	0.972617	1.290677	1.349135	1.427749	1.567076
...	...	...	...	...	...
4806	1.684946	0.785898	0.369465	0.112388	0.013702
4807	1.711220	0.813036	0.390414	0.110531	-0.007681
4808	1.677004	0.734032	0.309178	0.042255	-0.056038
4809	1.680179	0.728973	0.306184	0.041438	-0.055572
4810	1.682881	0.716645	0.283951	0.014608	-0.078540
[4811 rows x 5 columns]					

Figure 13. The smoothing factors of various years' yield curve.

We used K-means cluster and elbow method to be able to find the optimal number of clusters. According to the elbow method, we decided to use 6 clusters for our data.

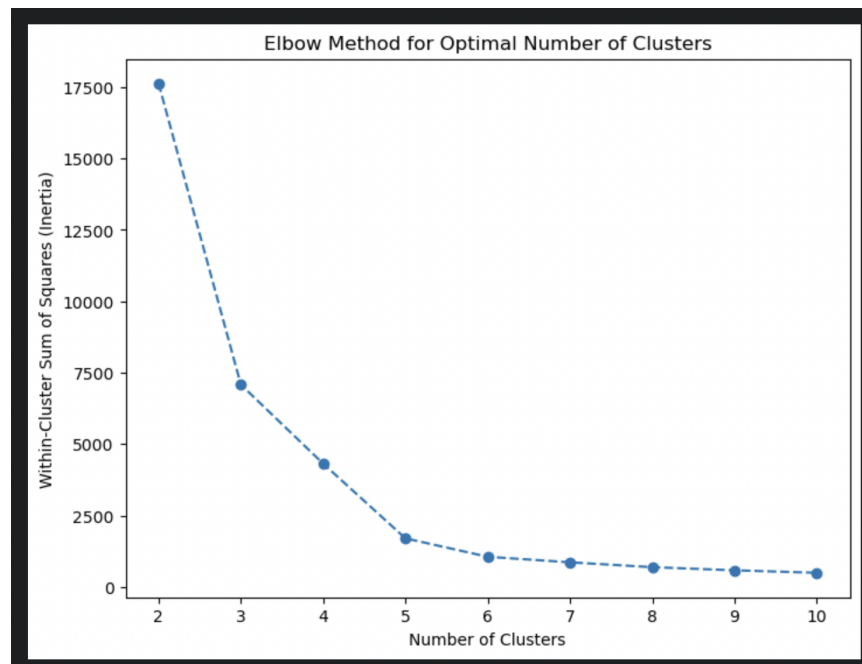


Figure 14. Sum of Squares under different numbers of clusters.

We first calculated the pairwise DTW distances using the fastdtw library. Then, we applied K-means clustering to DTW distances.

```
# Calculate pairwise DTW distances using fastdtw
dtw_distances = np.zeros((len(dtw_features), len(dtw_features)))
for i in range(len(dtw_features)):
    for j in range(i, len(dtw_features)):
        distance, _ = fastdtw(dtw_features[i], dtw_features[j])
        dtw_distances[i, j] = dtw_distances[j, i] = distance

# Apply K-Means clustering to DTW distances
kmeans = KMeans(n_clusters=optimal_num_clusters, random_state=42)
clusters = kmeans.fit_predict(dtw_distances)
```

Figure 15. Codes that utilized fastdtw library to calculate DTE distances.

Displaying our Cluster findings:

	Date	1Year	5Year	10Year	20Year	30Year	Cluster
0	2004-09-06	2.298838	3.457222	4.209220	4.779293	4.988680	1
1	2004-09-07	2.328891	3.479518	4.209626	4.768947	4.975495	1
2	2004-09-08	2.346666	3.507894	4.228419	4.776881	4.978894	1
3	2004-09-09	2.308988	3.430628	4.161872	4.733254	4.946545	1
4	2004-09-10	2.271566	3.374729	4.120981	4.702037	4.918530	1
...	...	...	...	...	...	...	...
4806	2023-06-21	3.400324	2.559223	2.516756	2.566076	2.469980	3
4807	2023-06-22	3.441493	2.604646	2.551937	2.562274	2.434478	3
4808	2023-06-23	3.384360	2.471258	2.413436	2.447870	2.356845	3
4809	2023-06-26	3.391265	2.466888	2.412622	2.450041	2.360039	3
4810	2023-06-27	3.395512	2.446554	2.375172	2.405090	2.322632	3

[4811 rows x 7 columns]

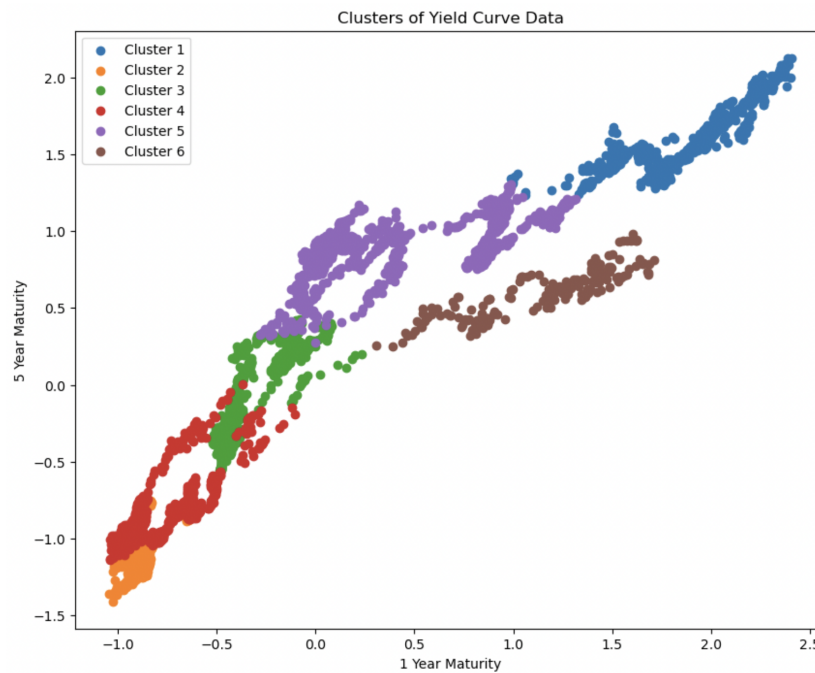


Figure 16. Cluster results.

Upon incorporating the clusters as a novel variable, it came to our attention that our model generates synthetic data for these clusters in floating-point numbers rather than discrete integer values ranging from 1 to 6.

This observation can be substantiated by referring to the last two lines in Figure 16. It's imperative to note that this discrepancy does not impinge upon the performance of other variables. However, it compromises the accuracy of the metrics, given that these are derived from the aggregate real and synthetic data for all variables. We recommend that the succeeding team assigned to this project address and enhance this particular facet.

Model id	seed	r2_tstr	r2_trtr	predictive_score	abs_metric	acf_id_lag=1	kurtosis	skew	cross_correl	sig_w1_metric
SigCWGAN	All Maturities Together Without Clustering	0,99673	0,99969	0,00296	0,05862	0,00360	0,01464	0,00459	0,09124	0,03333
SigCWGAN	All Maturities Together With Clustering	0,99374	0,99592	0,00217	0,09666	0,00476	0,02193	0,00985	0,34058	0,06181

Table 2. Comparison of SigCWGAN model having the clustering and without clustering.

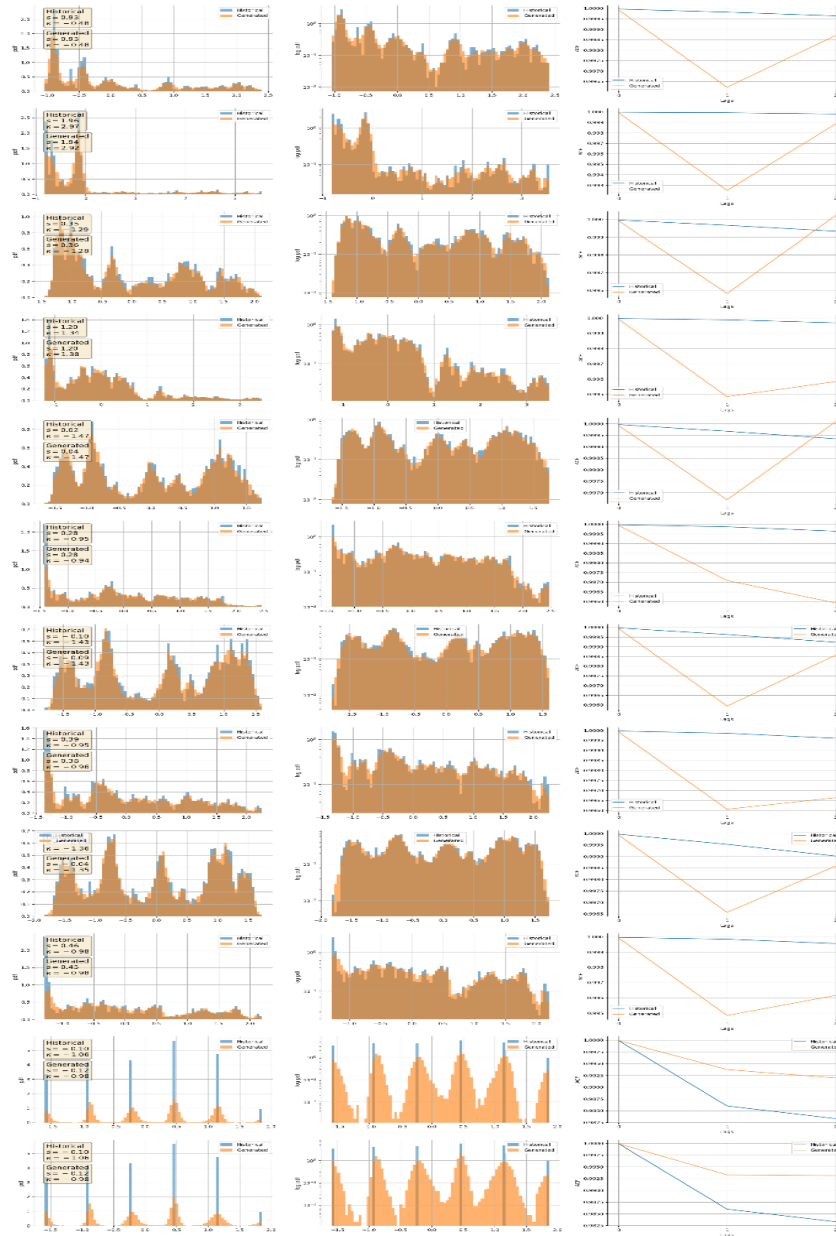


Figure 17. Results for Running with time series classification label.

5. Future Work

Instead of using the financial time series itself, using its signature to build a regime classification model may achieve more unexpectedly good results. Rough path theory is gradually gaining popularity as a method of capturing the intrinsic patterns of price movements[8]. Its essence is to reduce the dimensionality of the path's information by calculating the signature of a path[9], and use the signature instead of the path itself as the input features of the machine learning model. Since the signature of a rough path is unique[10], the signature well reflects the information of the original path. Moreover, most financial products' data are point-in-time data, not continuous, and highly volatile, which means its quadratic variance is often non-zero. Aiming at this feature, the path signature can well capture a continuous path and the characteristics of the non-zero quadratic variate of the original trajectory by leading--lagging deformation. Therefore, blending path signatures into regime classification may lead to unexpected surprises.

The recognition of regimes may feed back into the generation of synthetic GAN data. In recent years, with the sudden onset of risky events such as epidemics, identifying turning points in patterns has become an important proposition for financial researchers. Past researchers have made important contributions in regime classification. Kritzman et al. [11] characterized regimes by using Markov-switching models. Jiltsov [12] classified market regimes by introducing the Hidden Markov Models. Combining path signature, McIndoe et al. [13] proposed the Azran-Ghahramani clustering method in order to create a regime classification strategies. However, few articles have focused on the promising combination of signature of path, regime classification and neural network. Through rough path theory, the original data is transformed into a continuous sequence with nonzero quadratic features of the original trajectory. The GAN model is not only able to capture the distribution behind the paths well, but also can be used to classify the paths and realize the identification of turning points of financial asset price movements. The identification of these turning points may in turn help GAN to generate synthetic data since the regime itself also stands for a characteristic of data. Therefore, How to make the regime labels better help GAN models to generate synthetic data may be worth further study. What's more, in addition to the various small parameters in the GAN model, how to estimate the distance between distributions of signatures might be also worth studying. All in all, it's an unexplored territory that deserves to be plowed deeper.

6. Conclusion

In this project, the deployment and implementation of a virtual machine (VM) on the Google Cloud Platform (GCP) are discussed. The process involves setting up the VM instance, configuring the environment using Conda, and integrating GCP with Conda using a .yaml file. The specific details of creating the VM instance, specifying machine types, GPU configurations, and disk image settings are provided. The deployment process is supported by instructions for downloading and installing Miniconda, initializing Conda environments, and ensuring compatibility of package versions. The deployment also involves setting up a GitHub repository and loading data onto the VM.

The subsequent focus shifts to data loading and training a Generative Adversarial Network (GAN). The GAN training includes experimenting with multiple loss functions. The selection of the Mean Absolute Error (MAE) loss function is highlighted as the top performer among the evaluated alternatives. The implementation of a shell script for hyperparameter tuning is described, enhancing the efficiency of parameter optimization.

Results encompass various aspects, including project outcomes, yield data analysis and GAN performance evaluation. Key findings from analyzing SPX data reveal the superiority of the MAE loss function, and the analysis of yield data involves assessing various metrics and trends. The implementation of different GAN models across multiple yield curve maturities is discussed, with SigCWGAN emerging as the top-performing model. Time series classification techniques, including Dynamic Time Warping (DTW) and K-means clustering, are employed to classify yield data into clusters based on temporal features, with a focus on six clusters as determined by the elbow method. Cluster results are presented visually.

7. Acknowledgement

We would like to extend my profound gratitude to Professor Ali Hirsra for his invaluable guidance and unwavering support throughout the duration of this project. His insights and expertise greatly shaped the outcome and have been fundamental to my understanding of the subject.

We are equally thankful to TA Miao, whose patience and timely assistance made a significant impact on our work. Her suggestions and feedback were crucial in refining the project and ensuring its successful completion.

Additionally, we wish to express our heartfelt appreciation to the company representatives, Mr. Giuseppe Bonavolonta from EIB and Mr. Oleg Reichmann from ECB. Their industry perspectives, hands-on experience, and willingness to assist have enhanced the depth and relevance of this work.

The help, support, and assistance from each one of them have been paramount in making this endeavor a success. We are deeply grateful for their commitment and dedication.

8. References

- [1] Ni, H., Szpruch, L., Wiese, M., Liao, S., & Xiao, B. (2020). Conditional sig-wasserstein gans for time series generation. arXiv preprint arXiv:2006.05421.
- [2] Peyré, G., & Cuturi, M. (2019). Computational optimal transport: With applications to data science. *Foundations and Trends® in Machine Learning*, 11(5-6), 355-607.
- [3] Yoon, J., Jarrett, D., & Van der Schaar, M. (2019). Time-series generative adversarial networks. *Advances in neural information processing systems*, 32.
- [4] Esteban, C., Hyland, S. L., & Rätsch, G. (2017). Real-valued (medical) time series generation with recurrent conditional gans. arXiv preprint arXiv:1706.02633.
- [5] Hofert, M., Prasad, A., & Zhu, M. (2022). Multivariate time-series modeling with generative neural networks. *Econometrics and Statistics*, 23, 147-164.
- [6] H. Li, “Author Guidelines for CMPE 146/242 Project Report”, *Lecture Notes of CMPE 146/242*, Computer Engineering Department, College of Engineering, San Jose State University, March 6, 2006, pp. 1.
- [7] Niennattrakul, V., & Ratanamahatana, C. A. (2007, April). On clustering multimedia time series data using k-means and dynamic time warping. In *2007 International Conference on Multimedia and Ubiquitous Engineering (MUE'07)* (pp. 733-738). IEEE.
- [8] Lyons, T. (1998). Differential equations driven by rough signals. *Revista Matemática Iberoamericana*, Vol. 14(2), 215 – 310.
- [9] Levin, D., T. Lyons, and H. Ni (2016). Learning from the past, predicting the statistics for the future, learning an evolving system. Available at: <https://arxiv.org/abs/1309.0260>.
- [10] Boedihardjo, H., Geng, X., Lyons, T., & Yang, D. (2016). The signature of a rough path: uniqueness. *Advances in Mathematics*, 293, 720-737.
- [11] Kritzman, M., Page, S., & Turkington, D. (2012). Regime shifts: Implications for dynamic strategies (corrected). *Financial analysts journal*, 68(3), 22-39.
- [12] A. Jiltsov. Market regimes: How to spot them and how to trade them. 2020. FX Markets. <https://www.fx-markets.com/comment/7665631/market-regimes-how-to-spot-them-and-how-to-trade-them>.
- [13] McIndoe, C. A Data Driven Approach to Market Regime Classification.
- [14] Heber, G., Lunde, A., Shephard, N., & Sheppard, K. (2009). Oxford-Man Institute’s realized library. Version 0.1, Oxford&Man Institute, University of Oxford.